1. Fetch all products along with their supplier name (INNER JOIN).

```
# question 1
select products.product_id, products.product_name, suppliers.supplier_name
from products
inner join suppliers ON products.supplier_id = suppliers.supplier_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_id	product_name	supplier_name
1	iPhone 15	Reliance Digital
2	Samsung Gal...	Croma Electronics
9	Lenovo Laptop	Croma Electronics
3	Noise Smartw...	Flipkart Seller Hub
4	Boat Earbuds	Amazon India Vendor
7	Prestige Mixe...	Tata Cliq
8	Sony Bravia 5...	Vijay Sales
5	Kurta Set	Lifestyle Stores
6	Running Shoes	Lifestyle Stores
10	Philips Trimmer	Metro Electronics

2. Find all customers and their orders, even if they have not placed any (LEFT JOIN).

```
# question 2

select customers.name, orders.order_id
from customers
left join orders on orders.customer_id = customers.customer_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	order_id
Rohit Kumar	1
Rohit Kumar	31
Sneha Sharma	2
Sneha Sharma	26
Sneha Sharma	45
Amit Patel	3
Amit Patel	32
Amit Patel	46
Priya Reddy	4
Priya Reddy	33
Karan Singh	5
Karan Singh	27
Karan Singh	34
Karan Singh	47

Result 2

3. Get all suppliers and the products they supply, even if no products exist for a supplier (RIGHT JOIN).

```
# question 3

select suppliers.supplier_name, products.product_name
from suppliers
right join products on products.supplier_id = suppliers.supplier_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

supplier_name	product_name
Reliance Digital	iPhone 15
Croma Electronics	Samsung Gal...
Flipkart Seller Hub	Noise Smartw...
Amazon India Vendor	Boat Earbuds
Lifestyle Stores	Kurta Set
Lifestyle Stores	Running Shoes
Tata Cliq	Prestige Mixe...
Vijay Sales	Sony Bravia 5...
Croma Electronics	Lenovo Laptop
Metro Electronics	Philips Trimmer

4. Show all customers and all orders (FULL OUTER JOIN simulation using UNION)

```sql
# question 4
select customers.name, orders.order_id
from customers
left join orders on customers.customer_id = orders.customer_id

union

select customers.name, orders.order_id
from orders
left join customers on customers.customer_id = orders.customer_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	order_id
Rohit Kumar	1
Rohit Kumar	31
Sneha Sharma	2
Sneha Sharma	26
Sneha Sharma	45
Amit Patel	3
Amit Patel	32
Amit Patel	46
Priya Reddy	4
Priya Reddy	33
Karan Singh	5

Result 1

5. List all products priced between ₹5000 and ₹50,000 and supplied from "Mumbai".

```sql
select product_name, price,city
from products
inner join suppliers on products.supplier_id = suppliers.supplier_id
where price between 5000 and 50000
and suppliers.city = 'Mumbai' ;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_name	price	city

No answer is fulfilling the criteria so no output is generated.

6. Find the total number of orders placed by each customer and show only those who placed more than 2 .

```
194        AND s.city = 'Mumbai';
195
196    # question 6
197    SELECT customers.name,
198           COUNT(orders.order_id) AS NumberOfOrders
199    FROM orders
200    INNER JOIN customers
201        ON customers.customer_id = orders.customer_id
202    GROUP BY customers.name
203    HAVING COUNT(orders.order_id) > 2;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	NumberOfOrders
Sneha Sharma	3
Amit Patel	3
Karan Singh	4
Neha Verma	3
Arjun Mehta	4
Ritika Gupta	4
Vikram Joshi	5
Ananya Das	3
Suresh Iyer	3
Megha Kapoor	3
Ravi Shankar	4

7. Show each supplier’s total sales value (sum of quantity × price_each)?

```
# question no 7
SELECT s.supplier_name,
SUM(oi.quantity * oi.price_each) AS total_sales
FROM Suppliers s
JOIN Products p ON s.supplier_id = p.supplier_id
JOIN Order_Items oi ON p.product_id = oi.product_id
GROUP BY s.supplier_name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

supplier_name	total_sales
Reliance Digital	399995.00
Croma Electronics	795987.00
Flipkart Seller Hub	30994.00
Amazon India Vendor	17493.00
Tata Cliq	25996.00
Vijay Sales	239996.00
Lifestyle Stores	40983.00
Metro Electronics	10493.00

8. Find the average, highest, and lowest price of products in each category?

```
#question no 8
SELECT category,
AVG(price) AS avg_price,
MAX(price) AS max_price,
MIN(price) AS min_price
FROM Products
GROUP BY category;
```

Result Grid | Filter Rows: | Export: | Wrap

category	avg_price	max_price	min_price
Electronics	67749.000000	79999.00	55999.00
Wearables	3749.000000	4999.00	2499.00
Fashion	2499.000000	2999.00	1999.00
Home Appliances	6499.000000	6499.00	6499.00
Personal Care	1499.000000	1499.00	1499.00

9. Find the top 5 customers by total spending (ORDER BY SUM(total_amount) DESC LIMIT 5)?

```
#question no 9

SELECT customer_id, SUM(total_amount) AS total_spent
FROM Orders
GROUP BY customer_id
ORDER BY total_spent DESC
LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	total_spent
8	259996.00
9	255994.00
15	161997.00
2	156497.00
11	139996.00

10.Show the number of unique products ordered by each customer?

```sql
#question no 9
SELECT o.customer_id,
COUNT(DISTINCT oi.product_id) AS unique_products
FROM Orders o
JOIN Order_Items oi ON o.order_id = oi.order_id
GROUP BY o.customer_id;
```

Result Grid | Filter Rows: | Export: | Wrap C

customer_id	unique_products
1	2
2	2
3	2
4	2
5	3
6	2
7	3
8	2
9	3
10	2
11	3
12	3
13	3
14	3
15	2

11.Find customers who placed an order with an amount greater than the average order amount (subquery)?

```sql
#question no 11
SELECT name
FROM Customers
WHERE customer_id IN (
    SELECT customer_id
    FROM Orders
    WHERE total_amount > (SELECT AVG(total_amount) FROM Orders)
);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name
Rohit Kumar
Sneha Sharma
Karan Singh
Neha Verma
Arjun Mehta
Ritika Gupta
Vikram Joshi
Suresh Iyer
Megha Kapoor
Ravi Shankar
Aditya Jain

12. Find products that have never been ordered (subquery with NOT IN).

```sql
#question no 12
SELECT product_name
FROM Products
WHERE product_id NOT IN (
    SELECT DISTINCT product_id
    FROM Order_Items
);
```

Result Grid | Filter Rows: | Export:

name
Rohit Kumar
Sneha Sharma
Karan Singh
Neha Verma
Arjun Mehta
Ritika Gupta
Vikram Joshi
Suresh Iyer
Megha Kapoor
Ravi Shankar
Aditya Jain

13. List customers who ordered at least one product from the "Electronics" category.

```sql
#question no 13
SELECT DISTINCT name
FROM Customers
WHERE customer_id IN (
    SELECT o.customer_id
    FROM Orders o
    JOIN Order_Items oi ON o.order_id = oi.order_id
    JOIN Products p ON oi.product_id = p.product_id
    WHERE p.category = 'Electronics');
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name
Rohit Kumar
Sneha Sharma
Karan Singh
Neha Verma
Arjun Mehta
Ritika Gupta
Vikram Joshi
Ananya Das
Suresh Iyer

14. Get suppliers who provide products that have been ordered more than 100 times in total.

```
#question no 14
SELECT DISTINCT s.supplier_name
FROM Suppliers s
JOIN Products p
ON s.supplier_id = p.supplier_id
WHERE p.product_id IN (
    SELECT oi.product_id
    FROM Order_Items oi
    GROUP BY oi.product_id
    HAVING SUM(oi.quantity) > 100);
```

Result Grid | Filter Rows: | Export:

supplier_name

There are no supplier who provide products that have been ordered more than 100 times in total.

15. Find the most expensive product(s) using a subquery with MAX().

```
#question no 15
SELECT product_name
FROM Products
WHERE price = (SELECT MAX(price) FROM Products);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Cont

product_name
iPhone 15

16. Show orders placed by customers who live in either Mumbai, Delhi, or Bengaluru (IN operator).

```sql
#question no 16
SELECT *
FROM Orders
WHERE customer_id IN (
    SELECT customer_id
    FROM Customers
    WHERE city IN ('Mumbai', 'Delhi', 'Bengaluru'));
```

Result Grid | Filter Rows: | Edit: | Export/Impo

order_id	customer_id	order_date	total_amount	payment_mode
1	1	2024-01-05	79999.00	UPI
31	1	2024-03-16	1499.00	Cash
2	2	2024-01-08	74999.00	Credit Card
26	2	2024-03-05	6499.00	Credit Card
45	2	2024-04-13	74999.00	Net Banking
7	7	2024-01-22	6499.00	Credit Card
17	7	2024-02-16	6499.00	UPI
36	7	2024-03-26	55999.00	Cash
48	7	2024-04-19	2999.00	Cash
13	13	2024-02-08	4999.00	UPI
23	13	2024-02-28	7497.00	Credit Card
29	13	2024-03-12	55999.00	Net Banking
42	13	2024-04-07	2499.00	Net Banking
NULL	NULL	NULL	NULL	NULL

17. Show orders where payment mode is NOT UPI or Credit Card (NOT IN).

```
#question no 17
SELECT *
FROM Orders
WHERE payment_mode NOT IN ('UPI', 'Credit Card');
```

Result Grid | Filter Rows: | Edit: | Exp

order_id	customer_id	order_date	total_amount	payment_mode
3	3	2024-01-10	9998.00	Debit Card
5	5	2024-01-18	5997.00	Cash
8	8	2024-01-25	59999.00	Net Banking
10	10	2024-01-30	2998.00	Debit Card
12	12	2024-02-05	74999.00	Net Banking
14	14	2024-02-10	4998.00	Cash
18	8	2024-02-18	59999.00	Debit Card
20	10	2024-02-22	1499.00	Cash
22	12	2024-02-26	4999.00	Net Banking
25	15	2024-03-03	79999.00	Debit Card
28	9	2024-03-10	74999.00	Cash
29	13	2024-03-12	55999.00	Net Banking
31	1	2024-03-16	1499.00	Cash
33	4	2024-03-20	1999.00	Debit Card
35	6	2024-03-24	74999.00	Net Banking
36	7	2024-03-26	55999.00	Cash

18. Find customers who have no email address recorded (IS NULL).

```
#question no 18
SELECT name
FROM Customers
WHERE email IS NULL;
```

Result Grid | Filter Rows:

name

There are zero customers who have no email address recorded.

19. Show suppliers who are not from the same city as any customer (NOT IN subquery).

```
#question no 19
SELECT *
FROM Suppliers
WHERE city NOT IN (
    SELECT DISTINCT city
    FROM Customers);
```

Result Grid | Filter Rows:

supplier_id	supplier_name	contact_email	city
NULL	NULL	NULL	NULL

There are no suppliers who are not from the same city as any customer.

20. Get the latest 3 orders placed, skipping the first 2 (ORDER BY + LIMIT + OFFSET).

```
#question no 20
SELECT *
FROM Orders
ORDER BY order_date DESC
LIMIT 3 OFFSET 2;
```

Result Grid | Filter Rows: | Edit: | Expo

order_id	customer_id	order_date	total_amount	payment_mode
48	7	2024-04-19	2999.00	Cash
47	5	2024-04-17	7999.00	Credit Card
46	3	2024-04-15	5999.00	UPI
NULL	NULL	NULL	NULL	NULL